

LAB REPORT

Lab 3

File Integrity Verification & Hash Analysis

MD5 / SHA256

Course	Digital Forensics & Cybersecurity
Lab No.	03
Topic	File Integrity Verification using Hash Analysis
Tools	md5sum · sha256sum · Linux Terminal
Platform	Ubuntu / Kali Linux
Date	2025

*This lab was performed under the supervision of
Sir Sajid Hussain Raza*

LinkedIn Profile: <https://www.linkedin.com/in/sajid-hussain-raza/>

TABLE OF CONTENTS

1 Objective	3
2 Tools Required	3
3 Step-by-Step Procedure	4
3.1 Create a Sample File	4
3.2 Generate Original Hashes	4
3.3 Simulate File Tampering	5
3.4 Generate Post-Tampering Hashes	5
3.5 Compare Results	5
4 Hash Comparison Table	6
5 Forensic Analysis & Concepts	6
6 Conclusion	7

01 OBJECTIVE

To generate and compare file hashes (MD5 and SHA256) in order to verify file integrity before and after a file transfer or copying operation. This lab demonstrates a foundational technique used in digital forensics, evidence management, and chain-of-custody validation.

This lab is critical in real-world digital forensics because investigators must be able to prove that collected evidence has not been tampered with at any point during the investigation process.

02 TOOLS REQUIRED

Operating System	Linux — Ubuntu / Kali Linux
Hash Utility (MD5)	md5sum
Hash Utility (SHA256)	sha256sum
Interface	Terminal (Command-Line Interface)
Test Subject	Any sample file: image, PDF, or log file

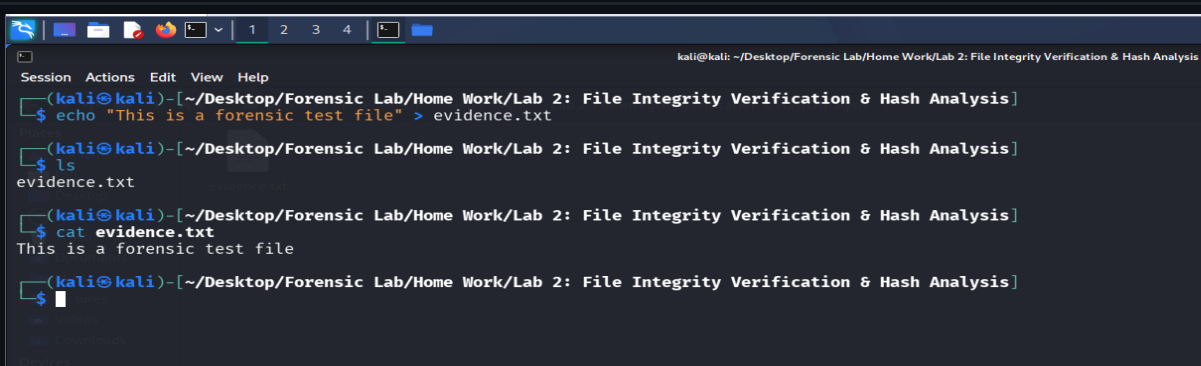
03 STEP-BY-STEP PROCEDURE

Step 1 — Create a Sample Evidence File

Begin by creating a simple text file that will serve as the forensic evidence file. This simulates an evidence artifact that needs to be protected and verified throughout the investigation.

COMMAND:

```
echo "This is a forensic test file" > evidence.txt
```



```
kali@kali: ~/Desktop/Forensic Lab/Home Work/Lab 2: File Integrity Verification & Hash Analysis
Session Actions Edit View Help
(kali@kali)-[~/Desktop/Forensic Lab/Home Work/Lab 2: File Integrity Verification & Hash Analysis]
$ echo "This is a forensic test file" > evidence.txt
(kali@kali)-[~/Desktop/Forensic Lab/Home Work/Lab 2: File Integrity Verification & Hash Analysis]
$ ls
evidence.txt
(kali@kali)-[~/Desktop/Forensic Lab/Home Work/Lab 2: File Integrity Verification & Hash Analysis]
$ cat evidence.txt
This is a forensic test file
(kali@kali)-[~/Desktop/Forensic Lab/Home Work/Lab 2: File Integrity Verification & Hash Analysis]
$
```

Step 2 — Generate Original Hash Values

Generate both MD5 and SHA256 cryptographic hash values for the original unmodified file. These values act as the *fingerprint* of the file and will be used as the reference baseline for comparison.

COMMANDS:

```
md5sum evidence.txt
sha256sum evidence.txt
```

```
(kali㉿kali)-[~/Desktop/Forensic Lab/Home Work/Lab 2: File Integrity Verification & Hash Analysis]
$ md5sum evidence.txt
sha256sum evidence.txt
b7793b115efed52dab4207b43c049f97  evidence.txt
335aaf2752deb85043fb515241e94311f455c5cf5a522f7dd57faf69765f409c  evidence.txt
(kali㉿kali)-[~/Desktop/Forensic Lab/Home Work/Lab 2: File Integrity Verification & Hash Analysis]
$
```

▲ Terminal Output — MD5 and SHA256 hash values of the original evidence.txt

Expected Hash Output:

MD5 Hash	b7793b115efed52dab4207b43c049f97
SHA256 Hash	335aaf2752deb85043fb515241e94311f455c5cf5a522f7dd57faf69765f409c

Save these baseline values to a reference file:

COMMANDS:

```
md5sum evidence.txt > original_hashes.txt
sha256sum evidence.txt >> original_hashes.txt
```

```

(kali@kali)-[~/Desktop/Forensic Lab/Home Work/Lab 2: File Integrity Verification & Hash Analysis]
$ md5sum evidence.txt > original_hashes.txt
sha256sum evidence.txt >> original_hashes.txt

(kali@kali)-[~/Desktop/Forensic Lab/Home Work/Lab 2: File Integrity Verification & Hash Analysis]
$ ls
evidence.txt  original_hashes.txt

(kali@kali)-[~/Desktop/Forensic Lab/Home Work/Lab 2: File Integrity Verification & Hash Analysis]
$ cat original_hashes.txt
b7793b115efed52dab4207b43c049f97  evidence.txt
335aaf2752deb85043fb515241e94311f455c5cf5a522f7dd57faf69765f409c  evidence.txt

(kali@kali)-[~/Desktop/Forensic Lab/Home Work/Lab 2: File Integrity Verification & Hash Analysis]
$

```

To direct input to this VM, move the mouse pointer inside or press Ctrl+G

▲ Terminal Output — Saving hash values to original_hashes.txt and verifying with cat

Step 3 — Simulate File Tampering

Modify the evidence file by appending additional data to simulate tampering or unauthorized alteration. Even a single byte change will produce a completely different hash output.

COMMAND:

```
echo "ADDING NEW DATA" >> evidence.txt
```

```

(kali@kali)-[~/Desktop/Forensic Lab/Home Work/Lab 2: File Integrity Verification & Hash Analysis]
$ echo "ADDING NEW DATA" >> evidence.txt

(kali@kali)-[~/Desktop/Forensic Lab/Home Work/Lab 2: File Integrity Verification & Hash Analysis]
$ ls
evidence.txt  original_hashes.txt

(kali@kali)-[~/Desktop/Forensic Lab/Home Work/Lab 2: File Integrity Verification & Hash Analysis]
$ cat evidence.txt
This is a forensic test file
ADDING NEW DATA

(kali@kali)-[~/Desktop/Forensic Lab/Home Work/Lab 2: File Integrity Verification & Hash Analysis]
$

```

▲ Terminal Output — Simulating tampering by appending data; cat confirms modification

Step 4 — Generate Post-Tampering Hash Values

Re-run the same hash commands on the now-modified file and record the new output values for comparison against the originals.

COMMANDS:

```
md5sum evidence.txt
sha256sum evidence.txt
```

```
(kali@kali)-[~/Desktop/Forensic Lab/Home Work/Lab 2: File Integrity Verification & Hash Analysis]
$ md5sum evidence.txt
sha256sum evidence.txt
d6e7243ebc2f36c57d8298033fd5eb5d  evidence.txt
97ec020cae4e614d3fd5d07d1dfc173b6e02e94180f4d6e360210c3fd47bc2b0  evidence.txt

(kali@kali)-[~/Desktop/Forensic Lab/Home Work/Lab 2: File Integrity Verification & Hash Analysis]
$
```

▲ Terminal Output — New MD5 and SHA256 hashes after file tampering (values differ from originals)

Post-Tampering Hash Output:

MD5 Hash	d6e7243ebc2f36c57d8298033fd5eb5d
SHA256 Hash	97ec020cae4e614d3fd5d07d1dfc173b6e02e94180f4d6e360210c3fd47bc2b0

Step 5 — Compare Results

Upon comparing the original and post-tampering hash values, two critical observations can be made:

- Hash values are **completely different** — even though only one line was appended.
- Even a **single character change** produces a totally new hash (avalanche effect).

04 HASH COMPARISON TABLE

STATE	MD5 HASH	SHA256 HASH
Original	b7793b115efed52dab4207b43c049f97	335aaf2752deb85043fb515241e94311 f455c5cf5a522f7dd57faf69765f409c
Modified	d6e7243ebc2f36c57d8298033fd5eb5d	97ec020cae4e614d3fd5d07d1dfc173b 6e02e94180f4d6e360210c3fd47bc2b0

■ Red values indicate tampered hash — mismatch detected. Green values represent the verified original baseline.

05 FORENSIC ANALYSIS & CONCEPTS

Cryptographic hashing is a cornerstone of digital forensics. The following concepts are illustrated by this lab:

■ Evidence Integrity

Hashing guarantees that a forensic copy is bit-for-bit identical to the original source. Any modification — however minor — is immediately detectable.

■ Court Evidence Validation

Hash values are presented in court to demonstrate that digital evidence has not been altered from the time of collection to the point of presentation.

■ Disk Imaging Verification

When creating forensic disk images (e.g., with dd or FTK Imager), hash values are computed before and after to confirm the image is an exact replica.

■ Malware Analysis

Security researchers use MD5/SHA256 hashes as unique identifiers for malware samples, enabling rapid lookup in threat-intelligence databases.

■ Chain of Custody

Every transfer of evidence is accompanied by its hash value, creating an unbroken, verifiable chain of custody from collection to court.

06 CONCLUSION

This lab successfully demonstrated the fundamental role of cryptographic hashing in digital forensics. By generating MD5 and SHA256 hash values for a file before and after modification, we observed that even the smallest change to a file produces an entirely different hash — confirming the **avalanche effect** inherent to

these algorithms.

The practical skills gained in this lab — including file creation, hash generation, baseline comparison, and tampering detection — are directly applicable in real forensic investigations, penetration testing engagements, and cybersecurity operations.

Hash-based verification remains one of the most reliable and legally accepted methods for maintaining evidence integrity, making it an indispensable tool in every cybersecurity professional's toolkit.

Supervised by: **Sir Sajid Hussain Raza**

<https://www.linkedin.com/in/sajid-hussain-raza/>